

Module 3

Lab - Conditions and Branching

Condition Statements

Comparison Operators

Code:

```
a = 5  
a == 6
```

Output:

False

Code:

```
i = 6  
i > 5
```

Output:

True

Code:

```
i = 2  
i > 5
```

Output:

False

Code:

```
i = 2  
i != 6
```

Output:

True

Code:

```
i = 6  
i != 6
```

Output:

False

Code:

```
"ACDC" == "The Bodyguard"
```

Output:

False

Code:

```
"ACDC" != "The Bodyguard"
```

Output:

True

Code:

```
'B' > 'A'
```

Output:

True

Code:

```
'BA' > 'AB'
```

Output:

True

Branching

Code:

```
age = 19  
#age = 18
```

```
if age > 18:  
    print("you can enter" )
```

```
print("move on")
```

Output:

you can enter
move on

Code:

```
age = 18  
# age = 19
```

```
if age > 18:  
    print("you can enter" )  
else:  
    print("go see Meat Loaf" )
```

```
print("move on")
```

Output:

go see Meat Loaf
move on

Code:

```
age = 18
```

```
if age > 18:  
    print("you can enter" )  
elif age == 18:  
    print("go see Pink Floyd")  
else:  
    print("go see Meat Loaf" )
```

```
print("move on")
```

Output:

```
go see Pink Floyd  
move on
```

Code:

```
album_year = 1983  
album_year = 1970  
  
if album_year > 1980:  
    print("Album year is greater than 1980")  
  
print('do something..')
```

Output:

```
do something..
```

Code:

```
album_year = 1983  
#album_year = 1970  
  
if album_year > 1980:  
    print("Album year is greater than 1980")  
else:  
    print("less than 1980")  
  
print('do something..')
```

Output:

```
Album year is greater than 1980  
do something..
```

Logical operators

Code:

```
album_year = 1980  
  
if(album_year > 1979) and (album_year < 1990):  
    print ("Album year was in between 1980 and 1989")  
  
print("")  
print("Do Stuff..")
```

Output:

```
Album year was in between 1980 and 1989
```

```
Do Stuff..
```

Code:

```
album_year = 1990  
  
if(album_year < 1980) or (album_year > 1989):  
    print ("Album was not made in the 1980's")
```

```
else:  
    print("The Album was made in the 1980's ")
```

Output:

Album was not made in the 1980's

Code:

```
album_year = 1983  
  
if not (album_year == 1984):  
    print ("Album year is not 1984")
```

Output:

Album year is not 1984

Quiz on Conditions

Dataset used:

Player Name	Sport	Achievements
Serena Williams	Tennis	23 Grand Slams
Lionel Messi	Soccer	7 Ballon d'Ors
Michael Phelps	Swimming	23 Gold Medals
Usain Bolt	Athletics	8 Gold Medals
Roger Federer	Tennis	20 Grand Slams
Cristiano Ronaldo	Soccer	5 Ballon d'Ors

Write a Python program to check if the player Lionel Messi has more than 10 achievements.

Code:

```
player_name = "Lionel Messi"  
sport = "Soccer"  
achievements = 7  
  
if achievements > 10:  
    print(f"{player_name} plays {sport} and has {achievements}  
achievements.")  
else:  
    print(f"{player_name} does not have more than 10 achievements.")
```

Output:

Lionel Messi does not have more than 10 achievements.

Write a Python program to check if a player belongs to the sport Tennis or has exactly 20 achievements.

Code:

```
player_name = "Roger Federer"  
sport = "Tennis"  
achievements = 20  
  
if sport == "Tennis" or achievements == 20:  
    print(f"{player_name} meets the criteria! They play {sport} and have  
{achievements} achievements.")
```

```
else:  
    print(f"{player_name} does not meet the criteria.")
```

Output:

Roger Federer meets the criteria! They play Tennis and have 20 achievements.

Write a Python program to check if a player has less than 10 achievements and does not play Soccer.

Code:

```
player_name = "Usain Bolt"  
sport = "Athletics"  
achievements = 8  
  
if achievements < 10 and sport != "Soccer":  
    print(f"{player_name} plays {sport} and has only {achievements}  
achievements.")  
else:  
    print(f"{player_name} does not meet the criteria.")
```

Output:

Usain Bolt plays Athletics and has only 8 achievements.

Lab completed - Conditions and Branching.

Lab - Loops

Range

Code:

```
# Use the range  
range(3)
```

Output:

```
range(0, 3)
```

What is for loop?

Code:

```
# For Loop example  
dates = [1982,1980,1973]  
N = len(dates)
```

```
for i in range(N):  
    print(dates[i])
```

Output:

```
1982  
1980  
1973
```

Code:

```
# Example of for loop
for i in range(0, 8):
    print(i)
```

Output:

```
0
1
2
3
4
5
6
7
```

Code:

```
# Example of for Loop, loop through list
for year in dates:
    print(year)
```

Output:

```
1982
1980
1973
```

Code:

```
# Use for loop to change the elements in list
squares = ['red', 'yellow', 'green', 'purple', 'blue']

for i in range(0, 5):
    print("Before square ", i, 'is', squares[i])
    squares[i] = 'white'
    print("After square ", i, 'is', squares[i])
```

Output:

```
Before square  0 is red
After square   0 is white
Before square  1 is yellow
After square   1 is white
Before square  2 is green
After square   2 is white
Before square  3 is purple
After square   3 is white
Before square  4 is blue
After square   4 is white
```

Code:

```
# Loop through the list and iterate on both index and element value
squares=['red', 'yellow', 'green', 'purple', 'blue']

for i, square in enumerate(squares):
    print(i, square)
```

Output:

```
0 red
```

```
1 yellow
2 green
3 purple
4 blue
```

What is while loop?

Code:

```
# While Loop Example
```

```
dates = [1982, 1980, 1973, 2000]
```

```
i = 0
```

```
year = dates[0]
```

```
while(year != 1973):
```

```
    print(year)
```

```
    i = i + 1
```

```
    year = dates[i]
```

```
print("It took ", i , "repetitions to get out of loop.")
```

Output:

```
1982
```

```
1980
```

```
It took  2 repetitions to get out of loop.
```

Controlling Loop Execution with break and continue

The break statement

Code:

```
for num in range(1, 10):
```

```
    if num == 5:
```

```
        print("Breaking the loop at:", num)
```

```
        break
```

```
    print(num)
```

Output:

```
1
```

```
2
```

```
3
```

```
4
```

```
Breaking the loop at: 5
```

The continue statement

Code:

```
for num in range(1, 6):
```

```
    if num == 3:
```

```
        continue
```

```
    print(num)
```

Output:

1
2
4
5

Code:

```
count = 0
while count < 10:
    count += 1
    if count == 3:
        continue # skip printing 3
    if count == 8:
        break # stop the loop when count is 8
    print(count)
```

Output:

1
2
4
5
6
7

Quiz on Loops

Write a for loop that prints out all the elements between -5 and 5 using the range function.

Code:

```
for i in range(-5, 6):
    print(i)
```

Output:

-5
-4
-3
-2
-1
0
1
2
3
4
5

Print the elements of the following list: Genres=['rock', 'R&B', 'Soundtrack', 'R&B', 'soul', 'pop']. Make sure you follow Python conventions.

Code:

```
Genres = ['rock', 'R&B', 'Soundtrack', 'R&B', 'soul', 'pop']
for Genre in Genres:
    print(Genre)
```

Output:

```
rock
R&B
Soundtrack
R&B
soul
pop
```

Write a for loop that prints out the following list: squares=['red', 'yellow', 'green', 'purple', 'blue']

Code:

```
squares=['red', 'yellow', 'green', 'purple', 'blue']
for square in squares:
    print(square)
```

Output:

```
red
yellow
green
purple
blue
```

Write a while loop to display the values of the Rating of an album playlist stored in the PlayListRatings list. If the score is less than 6, exit the loop. The list PlayListRatings is given by: PlayListRatings = [10, 9.5, 10, 8, 7.5, 5, 10, 10]

Code:

```
PlayListRatings = [10, 9.5, 10, 8, 7.5, 5, 10, 10]
i = 0
Rating = PlayListRatings[0]
while(i < len(PlayListRatings) and Rating >= 6):
    print(Rating)
    i = i + 1
    Rating = PlayListRatings[i]
```

Output:

```
10
9.5
10
8
7.5
```

Write a while loop to copy the strings 'orange' of the list squares to the list new_squares. Stop and exit the loop if the value on the list is not 'orange':

Code:

```
squares = ['orange', 'orange', 'purple', 'blue ', 'orange']
new_squares = []
i = 0
while(i < len(squares) and squares[i] == 'orange'):
    new_squares.append(squares[i])
    i = i + 1
```

```
print (new_squares)
```

Output:

```
['orange', 'orange']
```

Write a Python program using a for loop that prints numbers from 1 to 15 but:
Skips multiples of 3
Stops the loop if the number is greater than 12

Code:

```
for i in range(1, 16):  
    if i % 3 == 0:  
        continue # skip multiples of 3  
    if i > 12:  
        break     # stop if number > 12  
    print(i)
```

Output:

```
1  
2  
4  
5  
7  
8  
10  
11
```

Lab completed - Loops in Python.

Lab - Functions

What is a Function?

Code:

```
def add(a):  
    """  
    add 1 to a  
    """  
    b = a + 1  
    print(a, " if you add one", b)  
    return(b)
```

```
add(1)
```

Output:

```
1 if you add one 2
```

Code:

```
# Get a help on add function  
help(add)
```

Output:

Help on function add in module __main__:

```
add(a)
    add 1 to a
```

Code:

```
# Call the function add()
add(1)
```

Output:

```
1 if you add one 2
2
```

Code:

```
# Call the function add()
add(2)
```

Output:

```
2 if you add one 3
3
```

Code:

```
# Define a function for multiple two numbers
def Mult(a, b):
    c = a * b
    return(c)
    print('This is not printed')
```

```
result = Mult(12,2)
print(result)
```

Output:

```
24
```

Code:

```
# Use mult() multiply two integers
Mult(2, 3)
```

Output:

```
6
```

Code:

```
# Use mult() multiply two floats
Mult(10.0, 3.14)
```

Output:

```
31.400000000000002
```

Code:

```
# Use mult() multiply two different type values together
Mult(2, "The BodyGuard ")
```

Output:

```
'The BodyGuard The BodyGuard '
```

Variables

Code:

```
# Function Definition
def square(a):

    # Local variable b
    b = 1
    c = a * a + b
    print(a, "if you square + 1", c)
    return(c)
```

Output:

no output, this line just defines the function

Code:

```
# Initializes Global variable
x = 3
# Makes function call and return function a y
y = square(x)
y
```

Output:

```
3 if you square + 1 10
10
```

Code:

```
# Directly enter a number as parameter
square(2)
```

Output:

```
2 if you square + 1 5
5
```

Code:

```
# Define functions, one with return value None and other without return value
def M3():
    print('Hello')
```

```
def M31():
    print('Hello')
    return(None)
```

```
# See the output
M3()
```

Output:

```
Hello
```

Code:

```
# See the output
M31()
```

Output:

Hello

Code:

```
# See what functions returns are
print(M3())
print(M31())
```

Output:

```
Hello
None
Hello
None
```

Code:

```
# Define the function for combining strings
def con(a, b):
    return(a + b)

# Test on the con() function
con("This ", "is")
```

Output:

```
'This is'
```

Functions Make Things Simple

Block 1:

Code:

```
# a and b calculation block1
a1 = 4
b1 = 5
c1 = a1 + b1 + 2 * a1 * b1 - 1
if(c1 < 0):
    c1 = 0
else:
    c1 = 5
c1
```

Output:

```
5
```

Block 2:

Code:

```
# a and b calculation block2
a2 = 0
b2 = 0
c2 = a2 + b2 + 2 * a2 * b2 - 1
if(c2 < 0):
    c2 = 0
else:
    c2 = 5
c2
```

Output:

0

Code:

```
def Equation(a,b):  
    c=a+b+2*a*b-1  
    if(c<0):  
        c=0  
    else:  
        c=5  
    return(c)
```

Output:

no output, this line just defines the function

Block 3:

Code:

```
a1 = 4  
b1 = 5  
c1 = Equation(a1, b1)  
c1
```

Output:

5

Block 4:

Code:

```
a2 = 0  
b2 = 0  
c2 = Equation(a2, b2)  
c2
```

Output:

0

Pre-defined functions

Code:

```
# Build-in function print()  
album_ratings = [10.0, 8.5, 9.5, 7.0, 7.0, 9.5, 9.0, 9.5]  
print(album_ratings)
```

Output:

[10.0, 8.5, 9.5, 7.0, 7.0, 9.5, 9.0, 9.5]

Code:

```
# Use sum() to add every element in a list or tuple together  
sum(album_ratings)
```

Output:

70.0

Code:

```
# Show the length of the list or tuple
len(album_ratings)
```

Output:

8

In-Built functions

Code:

```
# Define a tuple
a = (1, 2)

# Pass the tuple to the sum function and store the result in a variable
c = sum(a)

# Print the result
print(f"The sum of the elements in the tuple {a} is {c}.")
```

Output:

The sum of the elements in the tuple (1, 2) is 3.

Code:

```
# Define a list
a = [1, 2]

# Pass the list to the sum function and store the result in a variable
c = sum(a)

# Print the result
print(f"The sum of the elements in the list {a} is {c}.")
```

Output:

The sum of the elements in the list [1, 2] is 3.

Using if/else Statements and Loops in Functions

Code:

```
# Function example
def type_of_album(album, year_released):

    print(album, year_released)
    if year_released > 1980:
        return "Modern"
    else:
        return "Oldie"

x = type_of_album("The BodyGuard", 1980)
print(x)
```

Output:

The BodyGuard 1980

Oldie

Code:

```
# Print the list using for loop
def PrintList(the_list):
    for element in the_list:
        print(element)

# Implement the printlist function
PrintList(['1', 1, 'the man', "abc"])
```

Output:

```
1
1
the man
abc
```

String comparison in Functions

Code:

```
#Compare Two Strings Directly using in operator
# add string
string= "The BodyGuard is the best album"

# Define a fuction
def check_string(text):

# Use if else statement and 'in' operator to compare the string
    if text in string:
        return 'String matched'
    else:
        return 'String not matched'

check_string("The BodyGuard is the best")
```

Output:

```
'String matched'
```

Code:

```
#Compare two strings using == operator and function
def compareStrings(x, y):
# Use if else statement to compare x and y
    if x==y:
        return 1
    else:
        return 0

compareStrings("BodyGuard", "BodyGuard")
```

Output:

```
1
```

Count the Frequency of Words Appearing in a String Using a Dictionary

Code:

```
# Python Program to Count words in a String using Dictionary
def freq(string):

    #step1: A list variable is declared and initialized to an empty list.
    words = []

    #step2: Break the string into list of words
    words = string.split() # or string.lower().split()

    #step3: Declare a dictionary
    Dict = {}

    #step4: Use for loop to iterate words and values to the dictionary
    for key in words:
        Dict[key] = words.count(key)

    #step5: Print the dictionary
    print("The Frequency of words is:",Dict)

#step6: Call function and pass string in it
freq("Mary had a little lamb Little lamb, little lamb Mary had a little
lamb.Its fleece was white as snow And everywhere that Mary went Mary went,
Mary went \
Everywhere that Mary went The lamb was sure to go")
```

Output:

```
The Frequency of words is: {'Mary': 6, 'had': 2, 'a': 2, 'little': 3, 'lamb':
3, 'Little': 1, 'lamb,': 1, 'lamb.Its': 1, 'fleece': 1, 'was': 2, 'white': 1,
'as': 1, 'snow': 1, 'And': 1, 'everywhere': 1, 'that': 2, 'went': 3, 'went,':
1, 'Everywhere': 1, 'The': 1, 'sure': 1, 'to': 1, 'go': 1}
```

Setting default argument values in your custom functions

Code:

```
# Example for setting param with default value
def isGoodRating(rating=4):
    if(rating < 7):
        print("this album sucks it's rating is",rating)
    else:
        print("this album is good its rating is",rating)
```

Output:

no output, this line just defines the function

Code:

```
# Test the value with default value and with input
isGoodRating()
isGoodRating(10)
```

Output:

```
this album sucks it's rating is 4
this album is good its rating is 10
```

Global variables

Code:

```
# Example of global variable
album = "The BodyGuard"

def printer(album):
    global internal_var
    internal_var= "Thriller"
    print(album,"is an album")

printer(album)
print(internal_var)
```

Output:

```
The BodyGuard is an album
Thriller
```

Note: Trying to access a variable that was only created inside a function without the global keyword (e.g. `printer1(internal_var1)`) raises a `NameError`, because a variable assigned inside a function is local by default and does not persist outside it.

Scope of a Variable

Code:

```
# Example of global variable
myFavouriteBand = "AC/DC"

def getBandRating(bandname):
    if bandname == myFavouriteBand:
        return 10.0
    else:
        return 0.0

print("AC/DC's rating is:", getBandRating("AC/DC"))
print("Deep Purple's rating is:",getBandRating("Deep Purple"))
print("My favourite band is:", myFavouriteBand)
```

Output:

```
AC/DC's rating is: 10.0
Deep Purple's rating is: 0.0
My favourite band is: AC/DC
```

Code:

```
# Deleting the variable "myFavouriteBand" from the previous example to
demonstrate an example of a local variable
del myFavouriteBand

# Example of local variable
def getBandRating(bandname):
    myFavouriteBand = "AC/DC"
```

```

    if bandname == myFavouriteBand:
        return 10.0
    else:
        return 0.0

print("AC/DC's rating is: ", getBandRating("AC/DC"))
print("Deep Purple's rating is: ", getBandRating("Deep Purple"))
print("My favourite band is", myFavouriteBand)

```

Output:

```

AC/DC's rating is:  10.0
Deep Purple's rating is:  0.0
NameError: name 'myFavouriteBand' is not defined

```

Code:

```

# Example of global variable and local variable with the same name
myFavouriteBand = "AC/DC"

```

```

def getBandRating(bandname):
    myFavouriteBand = "Deep Purple"
    if bandname == myFavouriteBand:
        return 10.0
    else:
        return 0.0

print("AC/DC's rating is: ", getBandRating("AC/DC"))
print("Deep Purple's rating is: ", getBandRating("Deep Purple"))
print("My favourite band is", myFavouriteBand)

```

Output:

```

AC/DC's rating is:  0.0
Deep Purple's rating is:  10.0
My favourite band is AC/DC

```

Collections and Functions

Code:

```

def printAll(*args): # All the arguments are 'packed' into args which can be
    treated like a tuple
    print("No of arguments:", len(args))
    for argument in args:
        print(argument)
#printAll with 3 arguments
printAll('Horsefeather', 'Adonis', 'Bone')
#printAll with 4 arguments
printAll('Sidecar', 'Long Island', 'Mudslide', 'Carriage')

```

Output:

```

No of arguments: 3
Horsefeather
Adonis
Bone
No of arguments: 4
Sidecar

```

Long Island
Mudslide
Carriage

Code:

```
def printDictionary(**args):  
    for key in args:  
        print(key + " : " + args[key])  
  
printDictionary(Country='Canada',Province='Ontario',City='Toronto')
```

Output:

Country : Canada
Province : Ontario
City : Toronto

Code:

```
def addItem(list):  
    list.append("Three")  
    list.append("Four")
```

```
myList = ["One","Two"]
```

```
addItem(myList)
```

```
myList
```

Output:

```
['One', 'Two', 'Three', 'Four']
```

Quiz on Functions

Come up with a function that divides the first input by the second input:

Code:

```
def div(a, b):  
    return(a/b)  
div(10, 2)
```

Output:

```
5.0
```

Use the function con for the following question.

Code:

```
def con(a, b):  
    return(a + b)
```

Can the con function we defined before be used to add two integers or strings?

Code:

```
con(2, 2)
```

Output:

4

Can the con function we defined before be used to concatenate lists or tuples?

Code:

```
con(['a', 1], ['b', 1])
```

Output:

```
['a', 1, 'b', 1]
```

Write a function code to find total count of word little in the given string:
"Mary had a little lamb Little lamb, little lamb Mary had a little lamb.Its
fleece was white as snow And everywhere that Mary went Mary went, Mary went
Everywhere that Mary went The lamb was sure to go"

Code:

```
def freq(string,passedkey):  
    words = []  
    words = string.split() # or string.lower().split()  
    Dict = {}  
    for key in words:  
        if(key == passedkey):  
            Dict[key] = words.count(key)  
    print("Total Count:",Dict)
```

```
freq("Mary had a little lamb Little lamb, little lamb Mary had a little  
lamb.Its fleece was white as snow And everywhere that Mary went Mary went,  
Mary went \n  
Everywhere that Mary went The lamb was sure to go","little")
```

Output:

```
Total Count: {'little': 3}
```

Lab completed - Functions in Python.

Lab - Objects and Classes

Introduction to Classes and Objects

Creating a Class

There are two classes used as examples: Circle (attributes radius, color) and Rectangle (attributes color, height, width).

Methods

Methods give a way to change or interact with an object. For example, add_radius(r) increases the radius of a circle by r.

Now we create a class Circle, but first we import a library to draw the objects:

Code:

```
# Import the library
import matplotlib.pyplot as plt
%matplotlib inline
```

Output:

no output, this line just imports the plotting library

Code:

```
# Create a class Circle
class Circle(object):

    # Constructor
    def __init__(self, radius=3, color='blue'):
        self.radius = radius
        self.color = color

    # Method
    def add_radius(self, r):
        self.radius = self.radius + r
        return(self.radius)

    # Method
    def drawCircle(self):
        plt.gca().add_patch(plt.Circle((0, 0), radius=self.radius,
fc=self.color))
        plt.axis('scaled')
        plt.show()
```

Output:

no output, this line just defines the class

Creating an instance of a class Circle

Code:

```
# Create an object RedCircle
RedCircle = Circle(10, 'red')
```

Output:

no output, this line just creates the object

Code:

```
# Find out the methods can be used on the object RedCircle
dir(RedCircle)
```

Output:

```
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattr__', '__getstate__', '__gt__',
 '__hash__', '__init__', '__init_subclass__', '__le__', '__lt__',
 '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
```

```
'__setattr__', '__sizeof__', '__str__', '__subclasshook__', '__weakref__',  
'add_radius', 'color', 'drawCircle', 'radius']
```

Code:

```
# Print the object attribute radius  
RedCircle.radius
```

Output:

```
10
```

Code:

```
# Print the object attribute color  
RedCircle.color
```

Output:

```
'red'
```

Code:

```
# Set the object attribute radius  
RedCircle.radius = 1  
RedCircle.radius
```

Output:

```
1
```

Code:

```
# Call the method drawCircle  
RedCircle.drawCircle()
```

Output:

draws a circle (visual plot, radius 1, colored red - not text output)

Code:

```
# Use method to change the object attribute radius  
print('Radius of object:',RedCircle.radius)  
RedCircle.add_radius(2)  
print('Radius of object of after applying the method  
add_radius(2):',RedCircle.radius)  
RedCircle.add_radius(5)  
print('Radius of object of after applying the method  
add_radius(5):',RedCircle.radius)
```

Output:

```
Radius of object: 1  
Radius of object of after applying the method add_radius(2): 3  
Radius of object of after applying the method add_radius(5): 8
```

Code:

```
# Create a blue circle with a given radius  
BlueCircle = Circle(radius=100)
```

Output:

no output, this line just creates the object

Code:

```
# Print the object attribute radius
BlueCircle.radius
```

Output:
100

Code:
Print the object attribute color
BlueCircle.color

Output:
'blue'

Code:
Call the method drawCircle
BlueCircle.drawCircle()

Output:
draws a circle (visual plot, radius 100, colored blue - not text output)

The Rectangle Class

Code:
Create a new Rectangle class for creating a rectangle object
class Rectangle(object):

```
    # Constructor
    def __init__(self, width=2, height=3, color='r'):
        self.height = height
        self.width = width
        self.color = color

    # Method
    def drawRectangle(self):
        plt.gca().add_patch(plt.Rectangle((0, 0), self.width,
self.height ,fc=self.color))
        plt.axis('scaled')
        plt.show()
```

Output:
no output, this line just defines the class

Code:
Create a new object rectangle
SkinnyBlueRectangle = Rectangle(2, 3, 'blue')

Output:
no output, this line just creates the object

Code:
Print the object attribute height
SkinnyBlueRectangle.height

Output:

3

Code:

```
# Print the object attribute width
SkinnyBlueRectangle.width
```

Output:

2

Code:

```
# Print the object attribute color
SkinnyBlueRectangle.color
```

Output:

'blue'

Code:

```
# Use the drawRectangle method to draw the shape
SkinnyBlueRectangle.drawRectangle()
```

Output:

draws a rectangle (visual plot, width 2, height 3, colored blue - not text output)

Scenario: Car dealership's inventory management system

Task-1. You are tasked with creating a Python program to represent vehicles using a class. Each car should have attributes for maximum speed and mileage.

Code:

```
class Vehicle:
    def __init__(self, max_speed, mileage):
        self.max_speed = max_speed
        self.mileage = mileage
```

Output:

no output, this line just defines the class

Task-2. Update the class with the default color for all vehicles, "white".

Code:

```
class Vehicle:
    color = "white"

    def __init__(self, max_speed, mileage):
        self.max_speed = max_speed
        self.mileage = mileage
```

Output:

no output, this line just defines the class

Task-3. Additionally, you need to create methods in the Vehicle class to assign seating capacity to a vehicle.

Code:

```
class Vehicle:
    color = "white"

    def __init__(self, max_speed, mileage):
        self.max_speed = max_speed
        self.mileage = mileage
        self.seating_capacity = None

    def assign_seating_capacity(self, seating_capacity):
        self.seating_capacity = seating_capacity
```

Output:

no output, this line just defines the class

Task-4. Create a method to display all the properties of an object of the class.

Code:

```
class Vehicle:
    color = "white"

    def __init__(self, max_speed, mileage):
        self.max_speed = max_speed
        self.mileage = mileage
        self.seating_capacity = None

    def assign_seating_capacity(self, seating_capacity):
        self.seating_capacity = seating_capacity

    def display_properties(self):
        print("Properties of the Vehicle:")
        print("Color:", self.color)
        print("Maximum Speed:", self.max_speed)
        print("Mileage:", self.mileage)
        print("Seating Capacity:", self.seating_capacity)
```

Output:

no output, this line just defines the class

Task-5. Additionally, you need to create two objects of the Vehicle class object that should have a max speed of 200mph and mileage of 20kmpl with five seating capacities, and another car object should have a max speed of 180mph and mileage of 25kmpl with four seating capacities.

Code:

```
# Creating objects of the Vehicle class
vehicle1 = Vehicle(200, 20)
vehicle1.assign_seating_capacity(5)
vehicle1.display_properties()

vehicle2 = Vehicle(180, 25)
vehicle2.assign_seating_capacity(4)
vehicle2.display_properties()
```

Output:

```
Properties of the Vehicle:  
Color: white  
Maximum Speed: 200  
Mileage: 20  
Seating Capacity: 5  
Properties of the Vehicle:  
Color: white  
Maximum Speed: 180  
Mileage: 25  
Seating Capacity: 4
```

Lab completed - Classes and Objects in Python.

Lab - Exception Handling

What is an Exception?

Definition

An exception is an error that occurs during the execution of code. This error causes the code to raise an exception and if not prepared to handle it will halt the execution of the code.

Examples

Code:

```
1/0
```

Output:

```
ZeroDivisionError: division by zero  
(ZeroDivisionError occurs when you try to divide by zero.)
```

Code:

```
y = a + 5
```

Output:

```
NameError: name 'a' is not defined  
(NameError -- in this case, it means that you tried to use the variable a  
when it was not defined.)
```

Code:

```
a = [1, 2, 3]  
a[10]
```

Output:

```
IndexError: list index out of range  
(IndexError -- in this case, it occurred because you tried to access data  
from a list using an index that does not exist for this list.)
```

Exception Handling

Try Except

A try except will allow you to execute code that might raise an exception and in the case of any exception or a specific one we can handle or catch the exception and execute specific code. This will allow us to continue the execution of our program even if there is an exception.

Try Except Specific

A specific try except allows you to catch certain exceptions and also execute certain code depending on the exception.

Try Except Specific Example

Code:

```
a = 1
```

```
try:
```

```
    b = int(input("Please enter a number to divide a"))
```

```
    a = a/b
```

```
    print("Success a-",a)
```

```
except ZeroDivisionError:
```

```
    print("The number you provided cant divide 1 because it is 0")
```

```
except ValueError:
```

```
    print("You did not provide a number")
```

```
except:
```

```
    print("Something went wrong")
```

Output (example run, dividing by 0):

The number you provided cant divide 1 because it is 0

Output (example run, entering a valid number like "2"):

Success a- 0.5

Try Except Else and Finally

else allows one to check if there was no exception when executing the try block. This is useful when we want to execute something only if there were no errors.

Practice Exercises

Exercise 1: Handling ZeroDivisionError

Imagine you have two numbers and want to determine what happens when you divide one number by the other. To do this, you need to create a Python function called `safe_divide`. You give this function two numbers, a 'numerator' and a 'denominator'. Use the user input method of Python to take the values.

The function should be able to do the division for you and give you the result. But here's the catch: if you try to divide by zero, the function

should catch that and tell you that it's not possible to divide by zero. Instead of showing an error, it should return None, and print "Error: Cannot divide by Zero."

Code:

```
def safe_divide(numerator, denominator):  
    try:  
        result = numerator / denominator  
        return result  
    except ZeroDivisionError:  
        print("Error: Cannot divide by Zero.")  
        return None
```

```
safe_divide(10, 2)
```

Output:

```
5.0
```

Code:

```
safe_divide(5, 0)
```

Output:

```
Error: Cannot divide by Zero.  
None
```

Exercise 2: Handling ValueError

Imagine you have a number and want to calculate its square root. To do this, you need to create a Python function. You give this function one number, 'number1'.

The function should generate the square root value if you provide a positive integer or float value as input. However, the function should be clever enough to detect the mistake if you enter a negative value. It should kindly inform you with a message saying, 'Invalid input! Please enter a positive integer or a float value.'

Code:

```
def square_root(number1):  
    try:  
        if number1 < 0:  
            raise ValueError  
        return number1 ** 0.5  
    except ValueError:  
        print("Invalid input! Please enter a positive integer or a float  
value.")  
        return None
```

```
square_root(16)
```

Output:

```
4.0
```

Code:

```
square_root(-9)
```

Output:

```
Invalid input! Please enter a positive integer or a float value.  
None
```

Exercise 3: Handling Generic Exceptions

Imagine you have a number and want to perform a complex mathematical task. The calculation requires dividing the value of the input argument "num" by the difference between "num" and 5, and the result has to be stored in a variable called "result".

You have to define a function so that it can perform this complex mathematical task. The function should handle any potential errors that occur during the calculation. To do this, you can use a try-except block. If any exception arises during the calculation, it should catch the error using the generic exception class "Exception" as "e". When an exception occurs, the function should display "An error occurred during calculation."

Code:

```
def calculate(num):  
    try:  
        result = num / (num - 5)  
        return result  
    except Exception as e:  
        print("An error occurred during calculation.")  
        return None
```

```
calculate(10)
```

Output:

```
2.0
```

Code:

```
calculate(5)
```

Output:

```
An error occurred during calculation.  
None
```

Lab completed - Exception Handling.